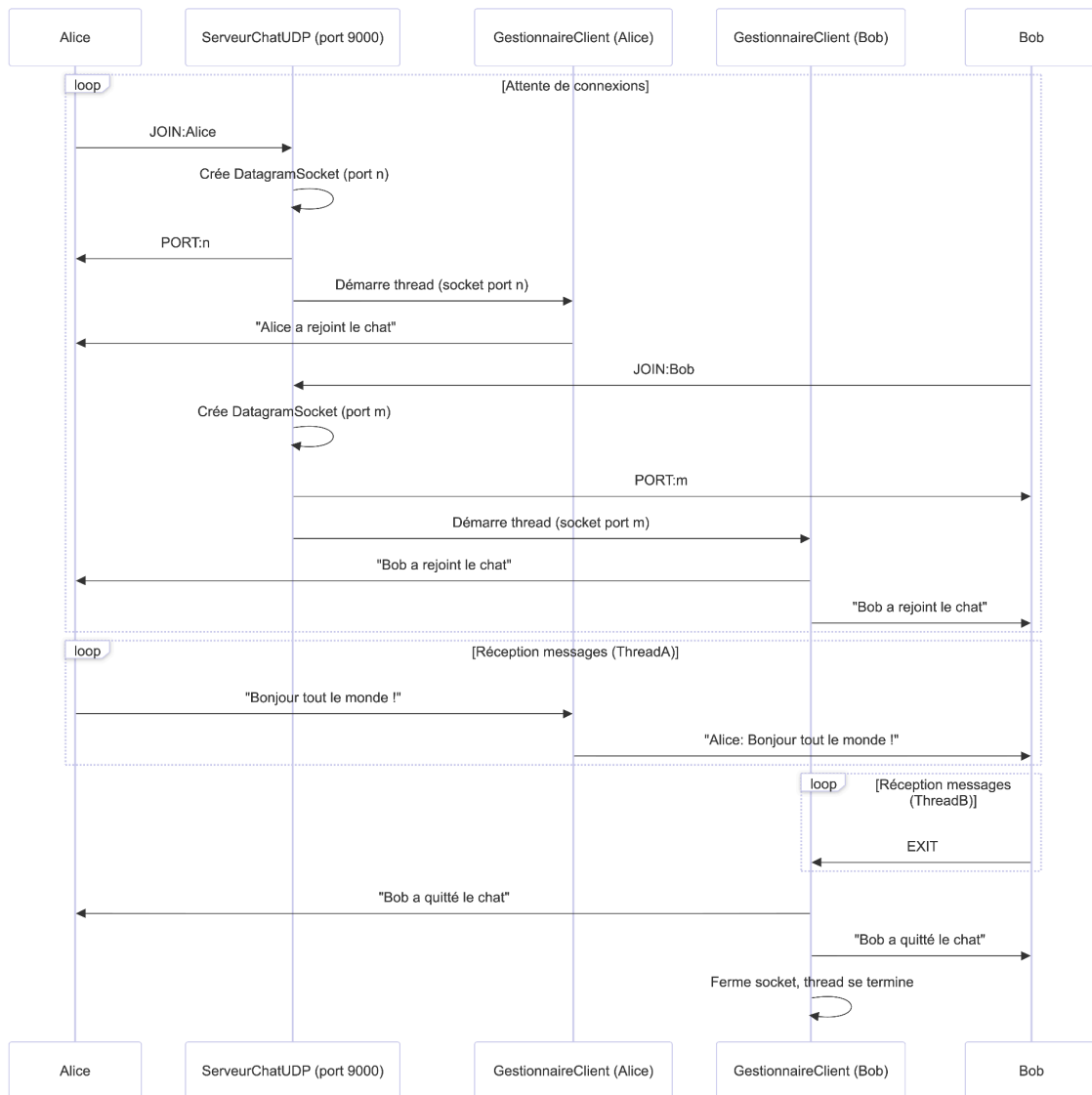


Exercice 1. Analyse et conception

Vous concevez une application de clavardage multi-utilisateurs en utilisant le protocole UDP. Afin d'éviter toute confusion entre les threads sur la réception, le serveur alloue une DatagramSocket dédiée à chaque client et communique au thread associé à un client le port correspondant. Le scénario nominal est le suivant :

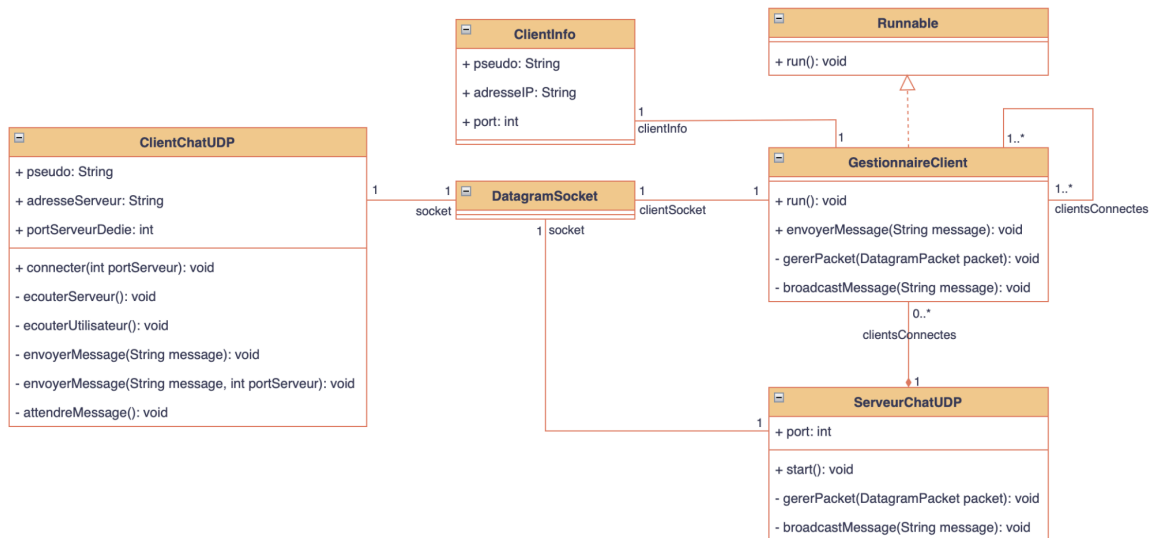
1. Alice envoie le message JOIN:Alice au serveur sur le port principal (9000);
2. le serveur enregistre Alice, crée une DatagramSocket sur un port libre et répond PORT: à Alice;
3. le serveur démarre un nouveau fil dédié à Alice, qui écoute exclusivement sur cette socket;
4. ce thread informe tous les clients connectés qu'Alice a rejoint le chat;
5. Bob envoie le message JOIN:Bob au serveur sur le port principal (9000);
6. le serveur enregistre Bob, crée une nouvelle DatagramSocket sur un port libre et répond PORT: à Bob;
7. le serveur démarre un nouveau fil dédié à Bob, qui écoute exclusivement sur cette socket;
8. ce thread informe tous les utilisateurs connectés que Bob a rejoint le chat;
9. Alice envoie le message « Bonjour tout le monde ! » sur le port dédié ;
10. le thread d'Alice diffuse ce message à tous les autres utilisateurs connectés, i.e. seulement Bob;
11. Bob envoie le message « EXIT » sur le port dédié ;
12. le thread de Bob supprime Bob et diffuse le message « Bob a quitté le chat » à tous les clients connectés, soit Alice et Bob;
13. le thread de Bob ferme sa socket dédiée et se termine

Q1. Donnez le diagramme de séquence UML du scénario nominal.



Q2. Donnez le diagramme de classes UML de l'application en faisant apparaître :

- la classe ClientChatUDP qui gère la saisie clavier et crée un thread d'écoute pour la réception;
- la classe ClientInfo qui encapsule les informations d'un client (pseudo, adresse IP, port);
- la classe GestionnaireClient qui implémente Runnable et gère les échanges avec un client;
- la classe ServeurChatUDP qui contient clients (la map partagée des clients) et dont le thread principal alloue, pour chaque nouveau client, uneDatagramSocketdédiée dont le port est communiqué au client avant de démarrer leGestionnaireCcorrespondant.



Exercice 2. Implémentation côté serveur

Q1. Implémentez la classe ClientInfo qui stocke le pseudo, l'adresse IP et le port d'un client. Munissez-la d'un constructeur et des accesseurs nécessaires.

Q2. Implémentez la classe GestionnaireClient qui implémente Runnable. Son constructeur prend en paramètre un objet ClientInfo, la DatagramSocket dédiée à ce client et la ConcurrentHashMap des clients connectés. La méthode run() doit :

1. diffuser le message de bienvenue à tous les clients;
2. recevoir en boucle les messages du client via la socket dédiée (ce thread est le seul à appeler receive() sur cette socket);
3. diffuser chaque message reçu à tous les autres clients;
4. traiter le cas EXIT : supprimer le client, diffuser le message de départ et fermer la socket dédiée.

Q3. Expliquez pourquoi l'utilisation d'une ConcurrentHashMap est préférable à une HashMap classique dans ce contexte.

Q4. Implémentez la classe ServeurChatUDP dont la méthode main() :

1. crée une DatagramSocket principale sur le port 9000;
2. attend en boucle les datagrammes entrants;
3. à la réception d'un message JOIN: :
 - (a) crée une DatagramSocket dédiée sur un port libre (new DatagramSocket(0));
 - (b) notifie le client du port alloué via le message PORT;
 - (c) enregistre le client et démarre un GestionnaireClient dans un nouveau thread en lui passant la socket dédiée.

Exercice 3. Implémentation côté client

Q1. Implémentez la classe ClientChatUDP dont la méthode main() :

1. demande à l'utilisateur de saisir son pseudo;
2. envoie JOIN: au serveur sur le port principal (9000);
3. attend la réponse PORT: du serveur et retient le port dédié pour la suite des échanges;
4. démarre un thread d'écoute qui reçoit et affiche en continu les messages entrants;
5. lit en boucle les messages saisis au clavier et les envoie au serveur sur le port dédié ;
6. envoie EXIT sur le port dédié et ferme la socket quand l'utilisateur tape exit.

Q2. Pourquoi est-il nécessaire de dédier un thread séparé à la réception des messages côté client?

Parce que `socket.receive()` est un appel bloquant, il suspend le thread jusqu'à la réception d'un paquet.

Sans thread séparé, le client pourrait soit :

- Lire la saisie clavier (mais pas recevoir de messages), soit
- Recevoir des messages (mais pas taper)

Jamais les deux en même temps. Le thread d'écoute permet de recevoir et affiche messages pendant que le thread principal lit l'entrée clavier. Sans cela, Alice ne pourrait pas voir les messages de Bob tant qu'elle est en train de taper.

Exercice 4. Tests et validation

Q1. Testez l'application en local en ouvrant 3 terminaux : un pour le serveur, un pour Alice et le dernier pour Bob. Vérifiez qu'un message envoyé par Alice est bien reçu par Bob et réciproquement.

Exercice 5. Pour aller plus loin

Q1. Ajoutez la commande /liste : lorsqu'un client l'envoie, le serveur lui retourne la liste des pseudos actuellement connectés.

Q2. Ajoutez les messages privés : un client peut envoyer /mp ; le serveur relaie le message uniquement au destinataire désigné, ou répond « Utilisateur inconnu » si le pseudo n'existe pas.

Q3. Modifiez le serveur pour qu'il détecte les clients inactifs depuis plus de 60 secondes (absence de message) et les déconnecte automatiquement après leur avoir envoyé TIMEOUT.